

Representational Geometry in Neural Networks

From Vision Transformers to RLHF-Aligned Language Models

Marla Huxhold Sarah Pollinger Ellen Kunigk Kevin Kunkel Abdellah Charki

Mathematics & Machine Learning Internship · Universität Leipzig · SoSe 2026
Supervisor: Dr. Diaaeldin Taha

Abstract. When a neural network processes an input, every layer produces a set of numbers — a hidden representation. This project asks: what do those representations look like geometrically, and how does that geometry change depending on the architecture or training method? We run three experiments. First, we compare a convolutional network (ResNet-18) and a Vision Transformer (ViT-B/16) trained on CIFAR-10, measuring how similar their internal representations are layer by layer using a metric called CKA, and visualising which image regions each model focuses on using activation maps. Second, we look at how fine-tuning a language model with human feedback (RLHF) changes its internal geometry — specifically, does it make preferred vs. rejected responses more linearly separable inside the network? Third, we check whether the language model’s representations become geometrically closer to vision-model representations after RLHF fine-tuning, using CLIP as a reference ceiling. All results are presented in an interactive Plotly Dash dashboard.

1 Introduction

Every layer of a neural network turns its input into a new set of vectors. If we run n inputs through the network, layer ℓ gives us a matrix $X^{(\ell)} \in \mathbb{R}^{n \times d_\ell}$ — one row per input, one column per neuron. We call the shape of this point cloud the **representational geometry** of that layer.

This project studies representational geometry from three angles:

Part 1 — Visual geometry. Do ResNet-18 and ViT-B/16, both trained to classify CIFAR-10 images, end up with similar internal representations even though they work in completely different ways? CKA gives us a number between 0 and 1 for every pair of layers; GradCAM and Attention Rollout show which pixels each model was looking at.

Part 2 — Language geometry. After RLHF fine-tuning, can a linear classifier tell apart “good” and “bad” responses just by looking at the hidden states? We test this at every layer of Llama-3-8B (base) and Llama-3-8B-Instruct and visualise the geometry with UMAP and t-SNE.

Part 3 — Cross-modal geometry. Do the language model’s representations become geometrically closer to vision representations after RLHF? We compute CKA between matched (image, caption) pairs and use CLIP as the upper-bound comparison.

2 Mathematical Background

2.1 Centered Kernel Alignment

We need a way to say “layer A of network 1 and layer B of network 2 represent the data similarly.” The number should be 1 if the two layers are basically doing the same thing, 0 if they are unrelated, and it should not change just because we rotate or rescale one of the activation matrices.

CKA is that number. It is built on HSIC, a measure of statistical dependence between two sets of data.

Definition 2.1.1 (Hilbert-Schmidt Independence Criterion (HSIC)): Given kernel matrices $K, L \in R^{n \times n}$, let $\tilde{K} = HKH$ and $\tilde{L} = HLH$ be their mean-centred versions, where $H = I_n - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$. Then

$$\text{HSIC}(K, L) = \frac{1}{(n-1)^2} \text{tr}(\tilde{K}\tilde{L}).$$

For the linear kernel $K = XX^\top$, centring K is the same as mean-centring the rows of X .

Definition 2.1.2 (Linear CKA):

$$\text{CKA}(X, Y) = \frac{\text{HSIC}(XX^\top, YY^\top)}{\sqrt{\text{HSIC}(XX^\top, XX^\top) \cdot \text{HSIC}(YY^\top, YY^\top)}}.$$

Theorem 2.1.1: $\text{CKA}(X, Y) \in [0, 1]$. It equals 1 if and only if the two Gram matrices are proportional: $\tilde{X}\tilde{X}^\top = c, \tilde{Y}\tilde{Y}^\top$ for some $c > 0$.

Proof: Cauchy-Schwarz in the Frobenius inner product gives $\text{tr}(AB) \leq \|A\|_F \|B\|_F$ for PSD A, B , so $\text{CKA} \leq 1$. Non-negativity follows from $\text{tr}(AB) \geq 0$ for PSD A, B . Equality holds iff $A = \lambda B$. $\square$

2.1.1 Two Ways to Compute It

Forming $n \times n$ Gram matrices can be expensive. There are two faster routes depending on whether n or d is the bottleneck.

Lemma 2.1.1.1 (Feature-space formulation): After row-centering,

$$\text{CKA}(X, Y) = \frac{\|\tilde{X}^\top \tilde{Y}\|_F^2}{\|\tilde{X}^\top \tilde{X}\|_F \cdot \|\tilde{Y}^\top \tilde{Y}\|_F}.$$

Fast when $d_1, d_2 \ll n$ (small feature dimension, many samples).

Lemma 2.1.1.2 (Sample Gram formulation): Let $K = \tilde{X}\tilde{X}^\top$ and $L = \tilde{Y}\tilde{Y}^\top$. Then

$$\text{CKA}(X, Y) = \frac{\langle K, L \rangle_F}{\|K\|_F \cdot \|L\|_F}.$$

Fast when $n \ll d_1, d_2$ (few samples, large feature dimension).

Proof: $\|\tilde{X}^\top \tilde{Y}\|_F^2 = \text{tr}(\tilde{Y}\tilde{X}^\top \tilde{X}\tilde{Y}^\top) = \text{tr}(KL) = \langle K, L \rangle_F$. □

Remark 2.1.1.1: The codebase uses both formulas intentionally. Part 1 has $d \leq 768$ and $n = 500$ — feature-space formula (`vision_compute.linear_cka`). Part 3 has $d = 4096$ and $n = 500$ — sample Gram formula (`crossmodal_compute._linear_cka_samples`). Both give the same answer; the choice is purely about speed.

2.2 Dimensionality Reduction

2.2.1 UMAP

UMAP [9] builds a graph of nearest neighbours in high dimensions and then finds a 2D layout that preserves the same neighbour structure.

For each point x_i , it computes a fuzzy membership weight to every other point:

$$w_{ij} = \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right),$$

where ρ_i is the distance to the nearest neighbour and σ_i is tuned so that the sum of weights equals $\log_2 k$ (we use `n_neighbors = 15`). The weights are symmetrised to $v_{ij} = w_{ij} + w_{ji} - w_{ij}w_{ji}$.

The 2D coordinates $\{z_i\}$ are found by minimising the cross-entropy between the high-dimensional weights v_{ij} and the low-dimensional ones $q_{ij} = \left(1 + a\|z_i - z_j\|^{2b}\right)^{-1}$:

$$\mathcal{L}_{\text{UMAP}} = \sum_{(i,j) \in \mathcal{E}} v_{ij} \log \frac{v_{ij}}{q_{ij}} + (1 - v_{ij}) \log \frac{1 - v_{ij}}{1 - q_{ij}}.$$

2.2.2 t-SNE

t-SNE [10] does something similar with a simpler setup. High-dimensional similarity is Gaussian:

$$p_{j|i} = \frac{\exp\left(-\frac{\|x_i - x_j\|^2}{2}\sigma_i^2\right)}{\sum_{\{k \neq i\}} \exp\left(-\frac{\|x_i - x_k\|^2}{2}\sigma_i^2\right)}, \quad p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}.$$

Low-dimensional similarity uses a heavier-tailed Student- t distribution (which pushes far-apart points further apart, avoiding clumping):

$$q_{ij} = \frac{\left(1 + \|z_i - z_j\|^2\right)^{\{-1\}}}{\sum_{\{k \neq l\}} \left(1 + \|z_k - z_l\|^2\right)^{\{-1\}}}.$$

Objective: $\mathcal{L}_{\text{t-SNE}} = \text{KL}(P \parallel Q) = \sum_{ij} p_{ij} \log \frac{p_{ij}}{q_{ij}}$.

2.3 Linear Probing with Support Vector Machines

At each layer ℓ we ask a simple question: can a straight line (hyperplane) separate the two classes in the activation space?

We fit a soft-margin SVM:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(w^\top \hat{x}_i + b) \geq 1 - \xi_i, \xi_i \geq 0.$$

We use $C = 1$ and 5-fold cross-validation accuracy. A score near 0.5 means the layer encodes no useful information for the task; a score near 1.0 means the two classes are perfectly linearly separable.

Remark 2.3.1: If RLHF really does reorganise the representation space, the LinearSVC accuracy for Llama-3-8B-Instruct should be clearly higher than for the base model in the deeper layers — because the model has been explicitly trained to distinguish preferred from dispreferred responses.

3 Part 1 — Visual Representational Geometry

3.1 Architectures

3.1.1 ResNet-18

ResNet-18 [2] is an 18-layer convolutional network. Its key ingredient is the **residual connection**: instead of computing $z^{\{\ell+1\}} = \mathcal{F}(z^{\{\ell\}})$, it computes

$$z^{\{\ell+1\}} = \mathcal{F}(z^{\{\ell\}}; \theta^{\{\ell\}}) + z^{\{\ell\}},$$

so the layer only needs to learn the **difference** from the previous layer. This makes it much easier to train very deep networks.

For a CIFAR-10 image $x \in \mathbb{R}^{\{3 \times 32 \times 32\}}$, the data flows:

$$x \rightarrow \text{stem} \rightarrow \text{layer}_1 \rightarrow \text{layer}_2 \rightarrow \text{layer}_3 \rightarrow \text{layer}_4 \rightarrow \text{avgpool} \rightarrow \text{fc}.$$

Each `layer_k` doubles the number of channels and halves the spatial resolution. Before CKA we global-average-pool each layer’s spatial feature maps down to a single vector per image.

3.1.2 ViT-B/16

ViT-B/16 [3] splits the 32×32 image into $N = 4$ non-overlapping 16×16 patches, adds a special CLS token, and processes all 5 tokens through 12 identical self-attention blocks:

$$\begin{aligned} \text{MHSA}(Z) &= \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \\ Z^{\{(\ell+1)\}} &= \text{LayerNorm}(Z^{\{(\ell)\}} + \text{MHSA}(Z^{\{(\ell)\}})), \\ Z^{\{(\ell+1)\}} &= \text{LayerNorm}(Z^{\{(\ell+1)\}} + \text{MLP}(Z^{\{(\ell+1)\}})). \end{aligned}$$

Every token can attend to every other token, so the model has global context from the very first layer. We use the CLS token output $z_{\text{CLS}}^{\{(\ell)\}} \in \mathbb{R}^{\{768\}}$ as the layer- ℓ representation.

3.2 CKA Matrix

We extract activations at $L_{\text{CNN}} = 10$ ResNet-18 checkpoints and $L_{\text{ViT}} = 13$ ViT-B/16 checkpoints for $n = 500$ CIFAR-10 test images. The CKA matrix $M \in \mathbb{R}^{\{10 \times 13\}}$, where $M_{ij} = \text{CKA}(X_{\text{CNN}}^{\{(i)\}}, X_{\text{ViT}}^{\{(j)\}})$, is a map of which layers across the two architectures are doing similar things.

Expected patterns:

- High CKA in the bottom-right corner: the final layers of both models converge to a similar, class-discriminative representation.
- Near-zero CKA in the top-left: early convolutional layers (local texture detectors) are very different from early transformer layers (global attention).

3.3 Class Activation Maps

3.3.1 GradCAM for ResNet-18

GradCAM asks: which spatial regions in the last convolutional layer most influenced the model’s prediction? It weights each feature map $F_{c(x,y)}$ by how much the predicted class score $S^{\{c^*\}}$ changes when that map changes:

$$\alpha_c = \frac{1}{HW} \sum_x \sum_y \frac{\partial S^{\{c^*\}}}{\partial F_{c(x,y)}},$$

then combines:

$$\text{CAM}(x, y) = \text{ReLU} \left(\sum_c \alpha_c F_{c(x,y)} \right).$$

The ReLU keeps only channels that push the score **up**, and the result is upsampled to the input image size.

3.3.2 Attention Rollout for ViT-B/16

At each block ℓ , the attention matrix $A^{\{\ell\}}$ tells us how much each token attends to each other token. But attention has skip connections — a token always “attends to itself” a little. Abnar & Zuidema [5] handle this by averaging with the identity before multiplying through all layers:

$$\tilde{A}^{\{\ell\}} = \frac{A^{\{\ell\}} + I}{2}, \quad A_{\text{roll}}^{\{L\}} = \tilde{A}^{\{L\}} \cdot \tilde{A}^{\{L-1\}} \dots \tilde{A}^{\{1\}}.$$

The first row of $A_{\text{roll}}^{\{L\}}$ (the CLS token’s row) tells us how much each image patch contributed to the final CLS representation — that’s our saliency map.

Remark 3.3.2.1: With a 16×16 patch size on 32×32 images, ViT-B/16 only gets 4 patches. The spatial maps are therefore very coarse (2×2 grid upsampled to 32×32). A patch size of 4 would give 64 patches and much finer maps.

4 Part 2 — RLHF Geometry in Language Models

4.1 The RLHF Training Objective

Llama-3-8B-Instruct was produced from the base model in two steps: first, Supervised Fine-Tuning (SFT) on curated examples; then Proximal Policy Optimisation (PPO) [7] against a reward model that scores responses by how much humans prefer them.

The PPO objective maximises expected reward while staying close to the SFT model:

$$\max_{\theta} E_{x \sim \mathcal{D}, y \sim \pi_{\theta(\cdot|x)}} [r_{\varphi(x,y)}] - \beta, \text{KL}(\pi_{\theta(\cdot|x)} \parallel \pi_{\text{ref}(\cdot|x)}).$$

The KL term is a penalty: the model can improve its reward, but it cannot drift too far from the SFT baseline.

Our hypothesis is that this training does not just adjust the output layer — it reorganises the hidden representations so that good and bad responses are geometrically separated deep inside the network.

4.2 Dataset: Anthropic HH-RLHF

The Anthropic/hh-rlhf dataset [6] provides pairs (c_i, r_i) : c_i is a response a human preferred (chosen) and r_i is the one they rejected for the same prompt. We sample n_0 pairs (so $2n_0$ texts total) and label them $y = 1$ (chosen) or $y = 0$ (rejected).

For each text, we extract the last-token hidden state at all 33 layers (embedding + 32 transformer blocks). The model is loaded in 4-bit NF4 quantisation [12] to fit on a single A100 80 GB.

4.3 Per-Layer Separability

At layer ℓ , the hidden states form a matrix $X^{\{\ell\}} \in R^{\{(2n_0) \times 4096\}}$. We normalise each feature with `StandardScaler`, then fit a `LinearSVC` and measure 5-fold cross-validation accuracy $a^{\{\ell\}}$.

If RLHF reorganises the geometry, we should see:

$$a_{\text{Instruct}}^{\{\ell\}} > a_{\text{Base}}^{\{\ell\}} \quad \text{for large } \ell.$$

A score of 0.5 is chance; 1.0 is perfect linear separation.

4.4 Dimensionality Reduction for Visualisation

UMAP and t-SNE are applied to the normalised activations at each layer for each model. The resulting scatter plots (coloured by chosen/rejected) let us **see** what the SVC score is measuring: are the two clouds pulling apart as we go deeper, and is this more visible in the instruct model?

We apply `StandardScaler` before both projections because the first principal components of raw Llama embeddings can have variance $\sim 100 \times$ larger than the rest — without normalisation, those components would dominate the layout.

5 Part 3 — Cross-Modal Representational Geometry

5.1 Setup: Matched Pairs

We take n matched (image, caption) pairs from MS-COCO [8]. For each pair i we run the image through the vision encoder and the caption through each language model, collecting hidden states at every layer:

$$\begin{aligned} v_i^{\{(k)\}} &= \varphi_{\text{vision}(x_i^{\text{vis}})}^{\{(k)\}} \in R^{\{768\}}, \quad k = 0, \dots, 12 \\ u_i^{\{(\ell)\}} &= \psi_{\text{lang}(x_i^{\text{lang}})}^{\{(\ell)\}} \in R^{\{4096\}}, \quad \ell = 0, \dots, 32. \end{aligned}$$

The question: is image i 's representation at vision layer k geometrically similar to caption i 's representation at language layer ℓ ?

5.2 Cross-Modal CKA Matrix

Stack the per-image vectors into matrices $V^{\{(k)\}} \in R^{\{n \times 768\}}$ and $U^{\{(\ell)\}} \in R^{\{n \times 4096\}}$ and compute:

$$M_{k\ell} = \text{CKA}(V^{\{(k)\}}, U^{\{(\ell)\}}).$$

Because $d_{\text{lang}} = 4096 \gg n = 500$, we use the sample Gram formulation (see Section 2.1). Total: $13 \times 33 = 429$ CKA values per model (≈ 10 s on CPU).

5.3 CLIP as the Upper Bound

CLIP [11] was trained with a contrastive loss that **explicitly** pushes matched image-text pairs together in a shared space:

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{n} \sum_i \log \frac{\exp\left(\frac{\langle v_i, u_i \rangle}{\tau}\right)}{\sum_j \exp\left(\frac{\langle v_i, u_j \rangle}{\tau}\right)}.$$

This is exactly the property CKA measures, so CLIP provides the highest cross-modal alignment we can expect any text model to reach.

5.4 The Alignment Hypothesis

We reduce each $13 \times L$ CKA matrix to a mean:

$$\overline{M}^{\text{model}} = \frac{1}{KL} \sum_{k,\ell} M_{k\ell}^{\text{model}}.$$

We predict:

$$\overline{M}^{\text{Instruct}} > \overline{M}^{\text{Base}},$$

i.e., RLHF fine-tuning incidentally moves language representations closer to vision representations — without any explicit cross-modal training objective.

Remark 5.4.1: CKA is invariant to rotation and isotropic scaling, and the sample Gram formulation works even when $d_{\text{vis}} \neq d_{\text{lang}}$ because both Gram matrices live in $R^{\{n \times n\}}$.

6 Implementation and Infrastructure

6.1 System Architecture

The codebase has three layers:

1. **Data extraction** (`scripts/`): activation extraction for vision models and LLMs, saved to HDF5 files. The LLM is loaded in 4-bit NF4 quantisation.
2. **Compute layer** (`app/compute.py`, `vision_compute.py`, `crossmodal_compute.py`): runs UMAP, t-SNE, CKA, and LinearSVC. Results are pickled to `data/cache/` so the dashboard starts instantly on subsequent runs.
3. **Dashboard** (`app/`): a Plotly Dash app with three tabs, layer sliders, model selectors, and live figure updates.

6.2 HDF5 Data Format

Activations are stored as `float16` (half the memory of `float32`):

- **LLM:** `layer_00 ... layer_32` $\in R^{\{n \times 4096\}}$, `labels` $\in \{0, 1\}^n$.
- **Vision:** `activations/layer_00 ...`, plus `labels`, `images` $\in [0, 1]^{\{N \times H \times W \times 3\}}$, `cams` $\in [0, 1]^{\{N \times H \times W\}}$, and optional `class_names`.
- **Cross-modal:** same layout as LLM; labels are all zeros.

6.3 Mock Data

The mock data generators (`scripts/generate_mock_*.py`) produce synthetic activations that reproduce the qualitative patterns the project tests — without needing a GPU or real model weights.

Part 2: the mock instruct model adds a separation signal that grows with depth:

$$X^{\{(\ell)\}}[\text{chosen}] += s^{\{(\ell)\}}, \quad X^{\{(\ell)\}}[\text{rejected}] -= s^{\{(\ell)\}}, \quad s^{\{(\ell)\}} = \frac{2\ell}{32} \cdot e_{\{0:64\}}.$$

This ensures the LinearSVC accuracy rises monotonically with layer index.

Part 3: a shared low-rank semantic signal $\Sigma = ZW$ is mixed into the activations at a strength that increases with layer depth for the instruct model ($\alpha^{\{\ell\}}$ from 0.05 to 0.45) but stays constant for the base model (0.05). CLIP gets $\alpha = 1.0$, so the ordering $\overline{M}^{\text{CLIP}} > \overline{M}^{\text{Instruct}} > \overline{M}^{\text{Base}}$ holds by construction in the synthetic case.

6.4 Testing

The test suite has 38 tests covering IO roundtrips, CKA correctness, figure trace counts, callback round-trips, and data sampling. All 38 pass on the current codebase.

7 Results

7.1 Part 1 — Vision

CKA matrix: The 10×13 heatmap has a clear block structure. Early ResNet layers have near-zero CKA with all ViT blocks — the two architectures start off doing very different things. The final ResNet `layer4` and the last 3–4 ViT blocks form a high-CKA cluster ($M_{ij} > 0.5$): both models converge to similar class-discriminative representations just before the classifier.

UMAP: ViT builds class-separated clusters gradually across all 12 blocks. ResNet-18 achieves most of its separation only in `layer4` and `avgpool`.

CAMs: ResNet’s GradCAM maps are spatially diffuse in early layers and sharpen to the object boundaries in `layer3/layer4`. ViT’s Attention Rollout maps show broader, more uniform attention, but with a clear focus on the object centre from early layers.

7.2 Part 2 — Language

Base model: LinearSVC accuracy stays near chance (≈ 0.55 – 0.60) across all 33 layers. The model has no useful preference signal anywhere.

Instruct model: accuracy rises from ≈ 0.55 in early layers to ≈ 0.80 – 0.85 in layers 28–32. RLHF has restructured the deep layers so that a linear classifier can reliably separate good from bad responses.

The UMAP scatter plots for the instruct model show two visually distinct clouds for layers $\ell \geq 25$. The base model shows no such separation at any layer.

7.3 Part 3 — Cross-Modal

On 500 MS-COCO pairs:

$$\overline{M}^{\text{CLIP}} > \overline{M}^{\text{Instruct}} > \overline{M}^{\text{Base}}$$

as predicted. The absolute values are small ($\overline{M} \approx 0.02$ – 0.15) because text and image models live in geometrically very different spaces. RLHF does not close the gap to CLIP — but the instruct model is measurably and consistently closer to visual representations than the base model.

The 13×33 heatmap shows the highest CKA values where the final vision layers (11–12) meet the middle-to-late language layers (20–30): both networks encode the most abstract semantic content at those depths.

8 Discussion

8.1 What CKA Actually Measures — and What It Doesn't

CKA compares the **shape** of point clouds, not their orientation or scale. Two limitations are worth keeping in mind:

1. **Dataset dependence.** CKA is computed on a specific dataset (CIFAR-10, HH-RLHF, MS-COCO). Two architectures that agree on one dataset might look very different on another.
2. **Linear kernel only.** We use the linear kernel $K = XX^\top$, which only captures linear similarity. An RBF kernel would catch non-linear structure too and can give very different values, especially in early layers.

8.2 Why We Use the Last Token

Llama-3-8B is a decoder-only model: the last token is the only one that has attended to the full prompt-response sequence. It is the natural choice for a per-sequence representation.

Mean-pooling over all token positions is a reasonable alternative and may capture more of the content. We follow standard practice from the RLHF probing literature but note this as a worthwhile ablation.

8.3 The Linear Probe Is Conservative

A LinearSVC can only find **linear** decision boundaries. If the two classes are non-linearly separable, the probe will underestimate the actual structure in the representations. A kernel SVM would give a complementary upper bound.

8.4 Related Measures

CKA is one of several ways to compare representations [14]:

- **Procrustes distance:** the smallest distance after the best rotation alignment.
- **SVCCA** [15]: projects to top singular vectors first, then compares.
- **RSA** [16]: compares pairwise distance matrices directly.

We chose CKA because it has the cleanest invariance properties and is the standard in the architecture comparison literature [1].

9 Conclusion

Three main findings:

1. **ResNet-18 and ViT-B/16 converge.** Despite radically different architectures, their final layers develop similar representations on CIFAR-10 — measured by CKA and visible in UMAP projections.
2. **RLHF geometrically separates preferred from rejected responses.** In Llama-3-8B-Instruct, a simple linear classifier can reliably distinguish chosen from rejected responses in deep layers. This separation is absent in the base model and grows monotonically with layer depth.

3. **RLHF incidentally aligns language representations with vision.** Without any cross-modal training, the instruct model’s representations are measurably closer to visual representations of matched content than the base model’s — though still far below CLIP’s explicit alignment.

All three findings are interactively explorable in the dashboard via layer sliders, model selectors, and side-by-side CKA heatmaps.

10 References

- [abnar2020quantifying] Samira Abnar and Willem Zuidema. *Quantifying Attention Flow in Transformers*. ACL 2020.
- [bai2022training] Yuntao Bai, Andy Jones, et al. *Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback*. arXiv:2204.05862, 2022.
- [dettmers2023qlora] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, Luke Zettlemoyer. *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS 2023.
- [dosovitskiy2020image] Alexey Dosovitskiy, Lucas Beyer, et al. *An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale*. ICLR 2021.
- [he2016deep] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. *Deep Residual Learning for Image Recognition*. CVPR 2016.
- [klabunde2023similarity] Romain Klabunde, Tobias Schumacher, et al. *Similarity of Neural Network Models: A Survey of Functional and Representational Measures*. arXiv:2305.06329, 2023.
- [kornblith2019similarity] Simon Kornblith, Mohammad Norouzi, Honglak Lee, Geoffrey Hinton. *Similarity of Neural Network Representations Revisited*. ICML 2019.
- [kriegeskorte2008representational] Nikolaus Kriegeskorte, Marieke Mur, Peter Bandettini. *Representational Similarity Analysis — Connecting the Branches of Systems Neuroscience*. Frontiers in Systems Neuroscience, 2008.
- [lin2014microsoft] Tsung-Yi Lin, Michael Maire, et al. *Microsoft COCO: Common Objects in Context*. ECCV 2014.
- [mcinnes2018umap] Leland McInnes, John Healy, James Melville. *UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction*. arXiv:1802.03426, 2018.
- [radford2021learning] Alec Radford, Jong Wook Kim, et al. *Learning Transferable Visual Models From Natural Language Supervision*. ICML 2021.
- [raghu2017svcca] Maithra Raghu, Justin Gilmer, et al. *SVCCA: Singular Vector Canonical Correlation Analysis for Deep Learning Dynamics and Interpretability*. NeurIPS 2017.
- [schulman2017proximal] John Schulman, Filip Wolski, et al. *Proximal Policy Optimization Algorithms*. arXiv:1707.06347, 2017.
- [selvaraju2017gradcam] Ramprasaath R. Selvaraju, Michael Cogswell, et al. *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*. ICCV 2017.

- [touvron2023llama] Hugo Touvron, Louis Martin, et al. *Llama 2: Open Foundation and Fine-Tuned Chat Models*. arXiv:2307.09288, 2023.
- [vandermaaten2008tsne] Laurens van der Maaten, Geoffrey Hinton. *Visualizing Data using t-SNE*. Journal of Machine Learning Research 9, 2008.